AirGated: A Prototype for Offline, Network-Bound Classroom Attendance with Device Keys and a Face-and-Blink Check

Aakshant Kumar
School of Engineering and Technology (UIET)
Chhatrapati Shahu Ji Maharaj University, Kanpur 208024, India
kumaraakshant2005@gmail.com

November 2026

Abstract

Attendance by roll call, QR code or GPS is defeated by cheap tricks: proxy answers, forwarded screenshots, spoofed coordinates. We describe AirGated, a research prototype that runs with no cloud service: a host on the classroom network creates the Wi-Fi network and a captive portal, and marks a student present only if, on one check-in, six gates pass: a fresh single-use server nonce; a server-measured round trip of at most 20 ms; a link-layer (MAC) address matching enrollment; an RSA signature from a non-extractable browser-held key over the nonce and the submitted frames; a blink liveness test with a face match against enrollment; and a limit of one roll number per MAC address in 60 s. We report the design, a defect history from three development phases, and what the same code base needed on two hosts (a Windows Mobile Hotspot and an Ubuntu laptop access point), isolating the small operating-system-specific surface: creating the access point, resolving an IP address to a MAC address, and the start order of the DNS responder. Field evidence is small: an August test (3 of 3 check-ins accepted, mean RTT 5.5 ms), a 30 September three-student trial with 17 logged events, informal testing with at least five people, and a Linux session. A loopback load test shows that the face engine, which analyses one clip at a time (about 2 s each), limits throughput to about 30 check-ins per minute (29 measured) well before the latency gate is affected. We propose treating a changed MAC as a re-registration trigger rather than a rejection when the device key and face still verify, and describe the next step: a dedicated, operating-system-independent access unit that bounds radio coverage to one room, removes the client cap of consumer hotspots, and lets the instructor run a class from a phone. AirGated is a prototype, not a production system, and we list what remains before it can be.

1 Introduction

Attendance is a trust problem, and each common technique has a known bypass: a friend answers "present"; a QR code on a projector is photographed and forwarded; a GPS geofence is defeated by mock-location settings; a login credential is copied to a friend's phone; a photograph of the absent student is held to a camera. AirGated is built as layers, each meant to block one of these (Table 1).

The design goals are: (i) *no cloud dependency* for attendance; (ii) *no app installation*, only a phone browser (each phone installs a classroom certificate once or accepts a browser warning); (iii) *binding to the physical phone and the physical person*; (iv) *inspectability*, with rejections written to an audit log and summarised on an instructor dashboard; and (v) *independence from any particular operating system or vendor*, since the intended end state is a dedicated access unit rather than a laptop.

Cheap trick	What stops it in AirGated
Friend answers "present"	Face must match the enrollment embedding
Forwarded QR code or screenshot	No QR code; each check-in signs a fresh single-use nonce
Fake GPS	GPS is not used; the phone must be on the classroom network with low server-measured latency
Logging in over a VPN or tunnel	The server times the round trip itself and rejects slow links
Credential copied to a friend's phone	The device key is non-extractable in the browser
Photo of the absent student	A random blink count (2 or 3) revealed only after the nonce is issued; this stops a still photo, not a recording (Section 10)
One phone checks in several students	One MAC address may check in one roll number per 60 s

Table 1: Attendance bypasses and the corresponding AirGated control.

Contributions.

- A six-gate check-in protocol in which the server, not the phone, measures proximity and makes every decision (Section 3).
- A defect history from three development phases, with root causes, showing which failures were logic errors and which were ordering and environment errors (Section 5).
- A host abstraction: five requirements any access host must meet, and the functions that were operating-system specific across a Windows and a Linux host (Section 6).
- A treatment of per-network randomized MAC addresses that keeps the device-binding gate while not costing a student the day when the network name or address setting changes, with failure messages that list every mismatching check (Section 8).
- A loopback load test identifying the face engine as the first scaling limit (Section 9), and a concrete plan for dedicated access hardware (Section 12).

2 Background and Related Work

Local captive portals. Phones decide whether a Wi-Fi network has internet access by requesting a known URL over plain HTTP and expecting a specific answer (for example HTTP 204). A different answer raises the OS sign-in window. AirGated answers every DNS name with the host's address and redirects probes to its landing page until the phone has checked in. The sign-in window is a restricted web view that generally cannot open the camera, so students are sent to a full browser for the check-in.

MAC address randomization. Mobile operating systems use random link-layer addresses to limit tracking; Martin et al. [1] study how this is implemented and when it fails. Android by default derives a persistent random address per network profile (SSID and security type) [6]; Apple's policy is different and rotates addresses in some cases [7]. The rules matter for any system that uses the MAC as an identity factor (Section 8).

Browser keys. The Web Cryptography API [2] lets a key be generated non-extractable and stored in IndexedDB; page script can ask for signatures but cannot read the key. Browsers expose this API and the camera only to secure origins, so the portal must be served over HTTPS even on a closed network.

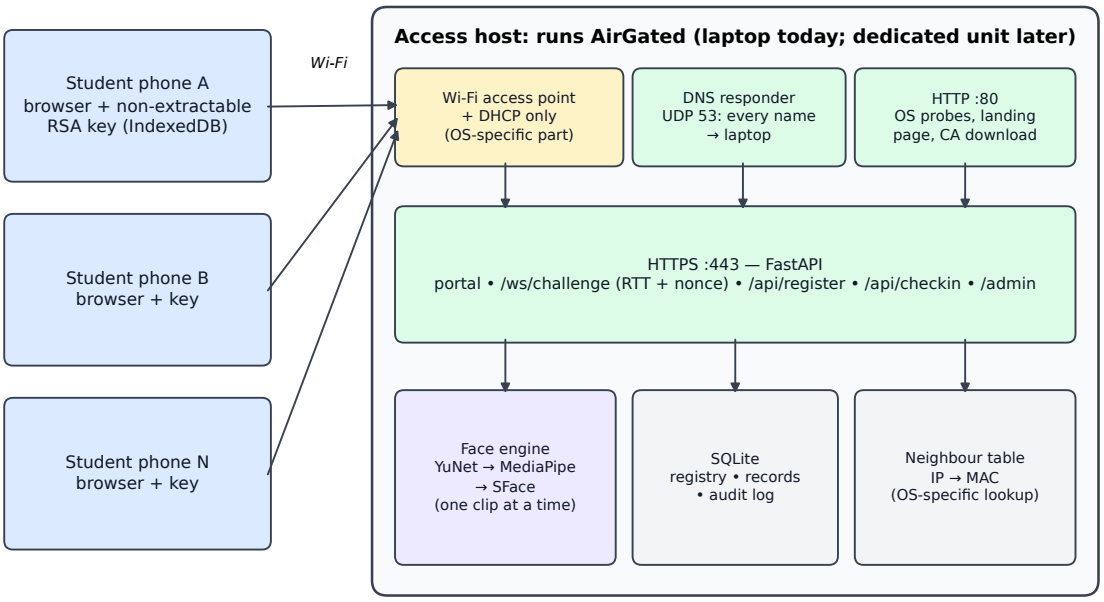


Figure 1: Architecture of the access host. Only the access point/DHCP block, the neighbour-table lookup and the DNS start order depend on the operating system.

Relay attacks on proximity. A rotating on-screen code resists a delayed screenshot but not a live relay: a student in the room points a camera at the projector and video-calls an accomplice, who reads the code from the live feed. Latency gating does not help, because the relay happens before any network request. We therefore treat radio containment as the primary presence boundary (Section 12). Ultrasonic proximity tokens were rejected early because attenuation rises with frequency and a laptop speaker is weak and non-directional, so back-row phones in a large room would not reliably hear it.

Face components. Detection uses YuNet [3], blink scoring uses face-blendshape output from MediaPipe's Face Landmarker [5], and identity embeddings use SFace [4] through OpenCV, with OpenCV's recommended cosine-similarity threshold of 0.363. All three models run on the host, offline.

3 System Design

3.1 Architecture

Figure 1 shows the *access host*: today a laptop, later a dedicated unit. It provides the Wi-Fi network and DHCP; a DNS responder that answers every query with the host's address; a plain-HTTP listener for OS probes, the landing page and the classroom certificate; and an HTTPS application (FastAPI) serving the portal, a challenge WebSocket, the registration and check-in APIs, and the instructor dashboard. State is kept in SQLite (registry, attendance records, audit log). The face engine and the neighbour table are reachable only from the application.

3.2 Enrollment

The browser generates an RSA-2048 key (RSASSA-PKCS1-v1.5, SHA-256) as non-extractable, requests a challenge, records a short clip while the student blinks, and signs the canonical message below with the new key as proof of possession. The server verifies the signature against the submitted public key, runs liveness, derives the face embedding (mean of up to 12 eyes-open

frames sampled at random across the clip, with every pair of sampled frames required to match), reads the MAC from its neighbour table, and stores key, MAC and embedding. The private key is persisted by the browser only after the server accepts, so a rejected enrollment leaves no orphaned key.

3.3 Challenge and signed message

Each registration or check-in consumes exactly one server-issued challenge: a 128-bit random nonce bound to purpose, roll number and client IP, valid for 120 s and deleted on first use. It carries the server-measured RTT and a random blink count $K \in \{2, 3\}$, revealed only after the nonce is issued. The phone signs

```
airgated|<purpose>|<nonce>|<roll>|<sha256(frames)>
```

where the last field is the SHA-256 digest of the concatenated JPEG frames, so neither the nonce nor the video can be swapped after signing.

3.4 The check-in gates

Figure 2 shows the pipeline. The latency gate uses the *minimum* of ten timed pings after three discarded warm-up pings. Phones in Wi-Fi power save wake only about every beacon interval (≈100 ms), so some pings are slow even beside the access point; congestion and sleep can only slow a ping, while a tunnel cannot make its fastest ping quicker than its real path. The threshold is 20 ms. The face gate requires one face in at least 60% of frames, at least K blinks (open→closed→open with hysteresis at 0.25/0.45 on the mean eye-blink score), and cosine similarity ≥ 0.363 to the enrolled embedding. Rejections are written to an audit table with event, roll number, IP, MAC and detail, and successful re-keys are shown on the dashboard.

3.5 Enrollment lifecycle

Early versions silently overwrote whichever roll was registered to a MAC, which downgrades the model to "trust the latest device on this MAC". The rules are now explicit: a device already enrolled to a different roll is refused (one phone, one student); a new roll enrolls normally; a roll already enrolled with a face re-keys only if the request is also signed by the enrolled key and the new face matches the stored one (audit event `reenroll_device`); anything else is refused; and a lost or replaced phone, or a cleared browser, needs an in-person instructor reset, which is audit-logged.

4 Roles and User Flows

AirGated has three roles. The *student* needs only a phone browser. The *instructor* starts and observes a session and handles exceptions. An *institution administrator* (not implemented) would manage rooms, retention and reporting.

Student. Figure 3 shows the flow: join the class network, enroll once (roll number, name, blink at the camera), then check in each class. A check-in ends in one of three ways: present, refused with a message naming every mismatching check, or an offer to re-register when only the MAC has changed. Re-registration needs the enrolled key, a fresh face match and then a normal check-in; a lost phone or cleared browser needs an instructor reset.

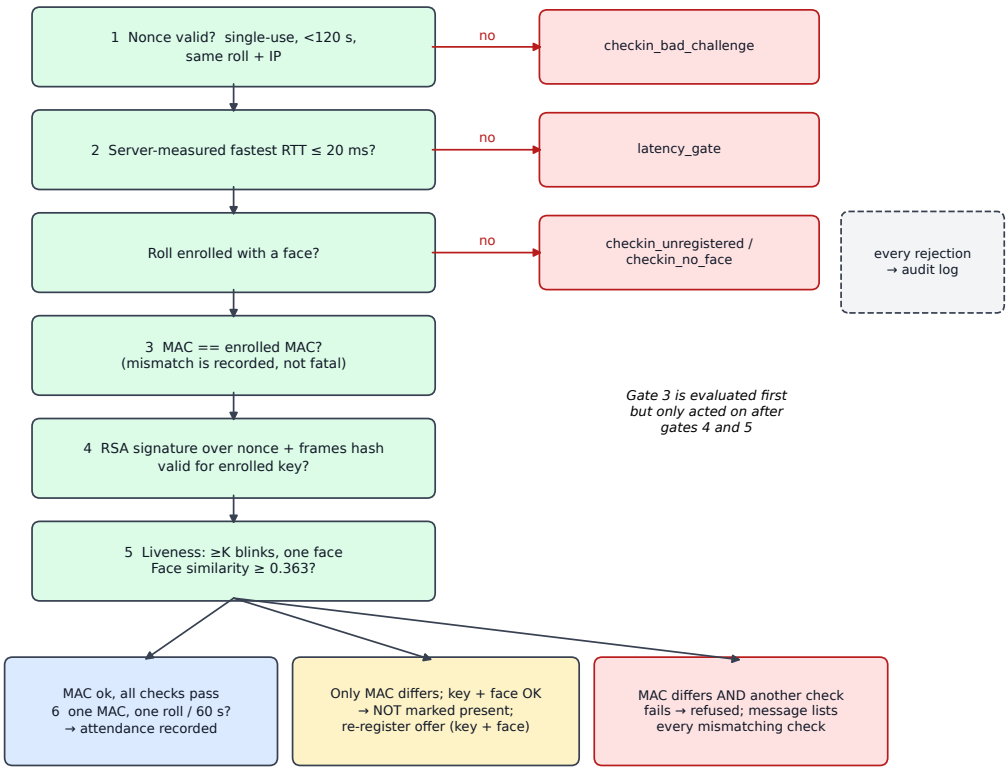


Figure 2: Check-in pipeline. A MAC mismatch is recorded at gate 3 but acted on only after the key and face checks, so a changed MAC alone leads to a re-registration offer instead of a rejection (Section 8).

Instructor. Figure 4 shows the flow. In the prototype the dashboard (present and absent lists, failure reasons by count, a seven-day trend, and a "needs attention" list of students who failed and were never marked present) is served only to the host itself, a security choice that also means the instructor must sit at the host. The dashed message in the figure, opening the dashboard from the instructor's own phone, is *proposed*: it needs an authenticated instructor channel, for example the instructor's phone enrolled with its own non-extractable key and the same face check, plus a rate limit. We have not built it.

5 Engineering History and Defects

The system went through three phases. The first used a cloud endpoint and browser geolocation with a Haversine distance, abandoned because client-side GPS is trivially spoofed. The second bound each phone to a browser-held key. The third hardened the network layer and added the face and blink gate. The defects found along the way are instructive because three of the eight were not logic errors (Table 2).

These three (route order, the port-53 race and the AP-isolation dependency) were ordering or environment failures that were hard to diagnose, which is why rejections are now audit-logged with a typed reason. The test suite exercises the real application in memory (HTTP and WebSocket) with a temporary database; each simulated phone owns a real RSA key and signs in the browser's exact format, and the face engine is replaced by a stub so tests choose what the camera "sees". To check that the tests detect defects, seven bugs were injected one at a time into a throwaway copy and the current suite was run against each (nonce not deleted after

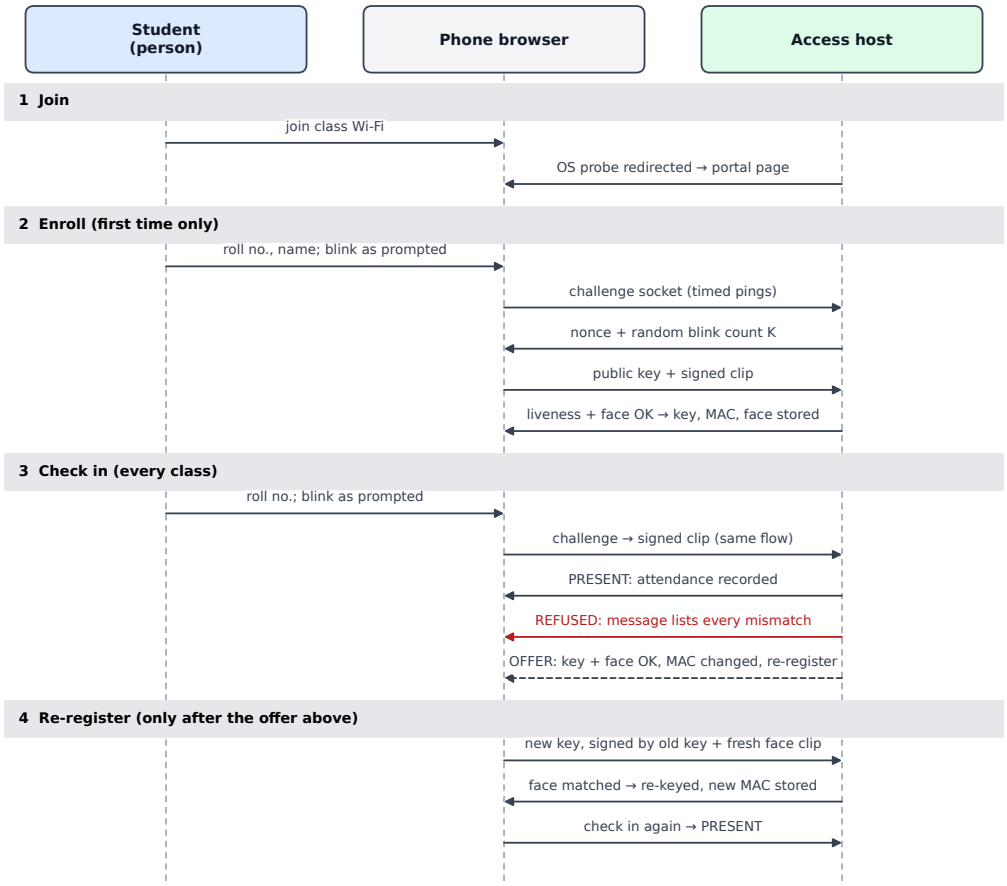


Figure 3: Student flow. The re-registration offer appears only when the device key and face verified and the MAC differs.

use, check-in face check removed, blink counter incrementing by two, probes answered 204 for everyone, MAC comparison disabled, re-key without the enrolled key, identity compared only with the clip mean); all seven were detected. The suite has 39 tests; the earlier five-bug run used a 31-test suite.

6 The Access Host: Requirements and Two Instantiations

Moving the system from a Windows laptop to an Ubuntu laptop exposed exactly which parts depend on the host. Everything else (routes, gates, face engine, database, dashboard) ran unchanged. We summarise this as five requirements:

H1 Gateway and no client isolation. The host must be the network's layer-2 gateway; otherwise it cannot see the phone's MAC.

H2 Owned DNS and DHCP. It must answer every DNS name with its own address and hand out leases that name itself as gateway and resolver, with no second DNS or DHCP service competing.

H3 Privileged ports. It must bind UDP 53 and TCP 80 and 443.

H4 Neighbour-table access. It must resolve a client IP to its MAC.

H5 Stable address. Its address on the class network must exist before the DNS responder binds, or the responder must bind a wildcard address.

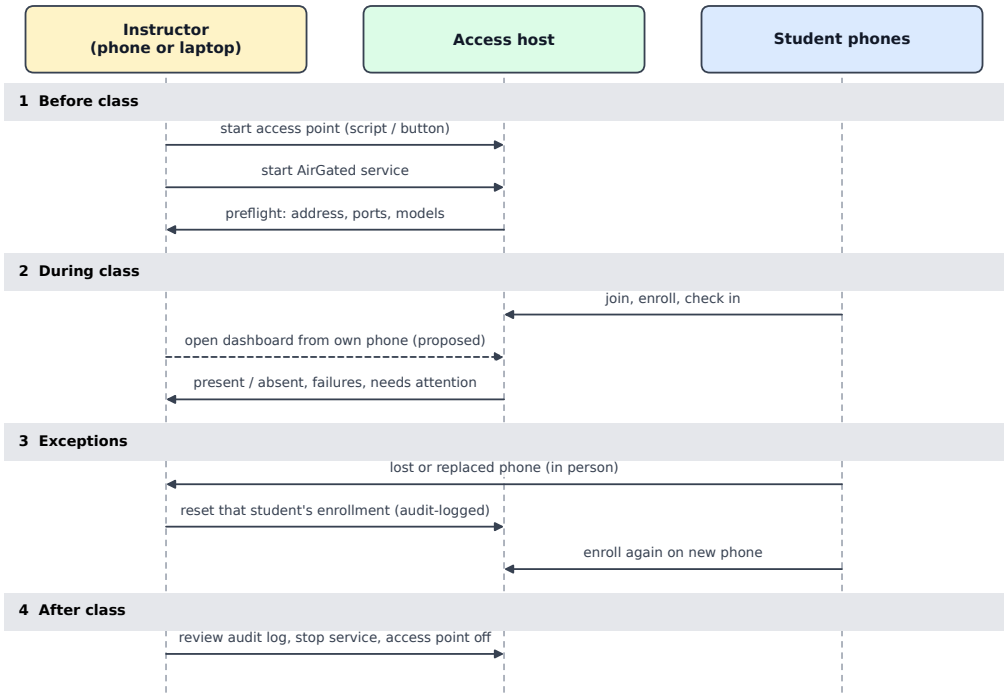


Figure 4: Instructor flow. The dashed arrow (dashboard from the instructor's phone) is proposed, not implemented.

Table 3 lists how each requirement was met. On Linux the port was done without editing the Windows entry point: a launcher imports the application and replaces two functions (IP-to-MAC lookup and the DNS responder), and a script creates the access point; all 31 original tests passed unchanged. Two Linux findings generalise. First, NetworkManager silently converted `ipv4.method manual` to `shared` for access-point profiles, which started its own `dnsmasq` on ports 53 and 67 and would have blocked the DNS responder; only reading the stored profile back and noticing `DHCP, IP range 192.168.137.10 - .254` in the journal revealed it. Only `disabled` is kept, so the address is added directly. Second, phones initially failed to join with the correct password; the journal changed from silence to `AP-STA-POSSIBLE-PSK-MISMATCH` after the WPA2/WPA3 transition mode was removed, and joining then worked after the saved network was forgotten and a simple password retyped. We did not isolate which of the two changes cured it.

Because the Linux card serves as the access point, the laptop has no internet while a session runs; attendance does not need it. The Windows cap of 8 clients can be raised by a registry override on paper, but the radio and driver still limit it in practice, and one student's whole flow (associate, lease, key generation, pings, sign, upload) was estimated at 15–30 s, so a class of 67 through 8 slots would need about 2–4 minutes even if each slot freed instantly; real disconnects hold slots longer, and we did not measure this. The Linux driver reports no station limit (`iw list` prints none), which does not prove there is none; we have not measured it.

7 Field Evidence

All evidence is small; none of it supports estimates of error rates.

August test (Windows). A phone never seen by the network joined without configuration, was redirected by the OS sign-in window, enrolled, and checked in three times: 3 of 3 accepted, server RTT 6.00, 5.01 and 5.44 ms (mean 5.48 ms), the MAC stable across the three requests.

Defect	Root cause	Fix
Private key exportable	The live inline portal generated the key with `extractable: true` and kept it in `localStorage`; the correct template was dead code	Non-extractable key in IndexedDB, saved only after server acceptance
Every registration and check-in returned `null`	A catch-all route was registered before the API routes; a bare `return` in a matched route sends an empty body	Register the catch-all last; regression test
No OS sign-in popup	Port-80 listener answered every probe with 204, the "internet works" answer	Redirect probes until the phone has checked in
DNS responder failed to start (Windows)	Internet Connection Sharing binds UDP 53 when the hotspot is switched on	Start the server first, then the hotspot
Replayable check-ins	Challenge was a hard-coded constant	Single-use server nonce, signature covers the frames
Spoofable latency	Phone reported its own RTT; only the last of three pings was timed	Server times ten pings after three warm-ups, uses the minimum
MAC unresolved on a home router	AP isolation stops client-to-client traffic, so the neighbour table is never populated	Requirement: the host must be the network's gateway (Section 6)
Silent re-binding	Re-registering overwrote the stored owner of a MAC	Re-key needs the enrolled key and a face match; one phone per roll; audited

Table 2: Defects found and their root causes.

30 September trial (Windows). Three students on their own phones about one metre from the host produced 17 logged events: 3 enrollments, 4 accepted check-ins and 10 rejections (Table 4). The MAC gate stopped a check-in sent from another student's phone; a face that did not match scored 0.204 against genuine scores of 0.715–0.879.

The two latency rejections occurred at about one metre, where propagation cannot explain 212.9 ms; power save did, and led to the minimum-of-ten estimator. Liveness rejections were the dominant friction: the blink counter saw fewer blinks than requested, plausibly because sampling at about 12 fps leaves a blink's closed phase in one to three frames, a single blurred frame can stay below the closed threshold, and students begin blinking during the countdown.

Wider testing. The author also tested with at least five people overall. Only logged sessions are reported as data.

Linux session (5 November 2026). The same application served two phones from the Ubuntu access point; one student's check-in, re-registration and later check-in appear in Section 8.

8 Randomized MAC Addresses and the Device-Binding Gate

After moving to the Linux access point, a previously enrolled student failed the MAC gate. The log contained the enrolled address `36:59:ce:...` (the locally administered bit is set, so it is a random address) and, on the new network, two other addresses appeared under this roll:

Requirement	Windows (Mobile Hotspot)	Ubuntu (NetworkManager AP)
Access point + DHCP	Provided by the OS (ICS) at `192.168.137.1`; capped at 8 clients by default	Created with `nmcli`; DHCP-only `dnsmasq` (`port=0`) so it does not own DNS
DNS responder	ICS binds UDP 53 when the hotspot starts, so start the server first	`systemd-resolved` holds 53 on loopback; bind `0.0.0.0` or the hotspot address, so the hotspot must exist first
IP to MAC	`arp -a`	`/proc/net/arp`, falling back to `ip neigh`; `arp` is not installed by default
Privileged ports	Administrator	Root or a lowered unprivileged-port floor

Table 3: The operating-system-specific surface. The order of starting the hotspot and the server is reversed between the two.

Observation	Value
Genuine face similarity (accepted)	0.715–0.879
Impostor face similarity (rejected)	0.204 (threshold 0.363)
RTT of accepted check-ins	3.9–7.2 ms
Latency false rejections (median-of-five estimator)	2 of 12 attempts (212.9 ms and 25.3 ms)
Liveness false rejections	4 of 12 check-ins, 2 of 5 enrollments
Cross-device check-in	Rejected by the MAC gate

Table 4: 30 September 2026 trial. Four genuine scores from three students and one impostor score are far too few to calibrate thresholds.

`00:9c:c0:...`, whose key was never checked, and a random one, `e6:4d:43:...`, which later verified with the enrolled key.

Observation. Over six phones and three to four days on the Windows hotspot, with no MAC-related setting changed, each phone kept the same address on a given network; on Linux one phone presented the same random address across repeated check-ins. The enrolled address stopped matching when the network name changed. This agrees with Android's documented behaviour, where the persistent address is derived from the network profile (SSID and security type) and survives forgetting and re-adding the network [6]. Apple documents a different policy: by default a fixed private address on WPA2-or-stronger networks and one that rotates every two weeks on weak or open networks; a new address after network settings are reset; from iOS 18, forgetting the network also discards the address; and, before iOS 18, a new address after six weeks without joining [7]. Our observation was informal and uninstrumented and we did not record phone models, so it should not be assumed to hold for iPhones. An earlier report had predicted random addresses would change between reconnects and listed disabling private addresses as a deployment prerequisite; the observation weakens that concern but does not exclude rotation.

Problem. Rejecting at gate 3 before checking the key and face made a changed network name or address setting cost a student the day, and the message ("Hardware MAC address signature mismatch") said nothing about what to do.

Change. A MAC difference no longer ends the check-in; the signature and the face and blink analysis still run. If only the MAC differs, attendance is *not* marked, the server logs `mac_changed_verified`, and the page offers re-registration, which reuses the re-key path (enrolled

N	RTT p95 / max (ms)	Gate rejections	Response p50 (s)	Response p95 (s)	Check-ins/min
1 (lone)	0.2 / 0.2	0/1	2.3	2.3	16.3
5	0.8 / 0.9	0/5	5.2	6.8	27.8
10	0.5 / 0.6	0/10	9.3	16.2	29.5
20	0.6 / 0.8	0/20	19.1	35.1	29.3

Table 5: Loopback load test; one run per level (raw output in the repository). An unqueued analysis takes 2.02 s. Rates include the start spread and the one-second capture wait, so the $N = 1$ figure mostly reflects that overhead.

key plus face match). If the MAC differs and anything else fails, the check-in is refused with a message that lists every mismatching check, for example "Not matching: 1. Wi-Fi hardware (MAC) address differs ... 2. Face does not match the enrolled student", and no re-registration is offered. If the MAC matches, behaviour is unchanged. A valid signature from a non-extractable key shows the enrolled browser is answering, and the face match shows the enrolled person holds it, so a changed address alone is not evidence of fraud. The MAC still drives the 60-second one-phone-one-student gate. The cost is that the MAC is no longer a hard factor; this is a trade-off whose rationale is that the key and face already bind the phone and the person. A student can also change the MAC without root, which defeats the one-phone-one-student and 60-second rules; the face check is then the remaining barrier.

Live confirmation. The audit log recorded `mac_changed_verified` (face 0.674), a failed re-registration (asked for 2 blinks, saw 0), `reenroll_device` (face 0.618), then an accepted check-in (face 0.582, three blinks, RTT 3.7 ms), all above the 0.363 threshold. This session predates the enrolled-key requirement for re-keying and the pairwise frame check; those two are covered only by automated tests. Four new tests cover the verified, wrong-face, foreign-key and too-few-blinks cases.

9 Load Test

The latency gate and the face engine are the two components most likely to degrade in a full class. A load test starts a private server on loopback with a scratch database and inserts fake enrollments. N simulated students each open the challenge WebSocket (answering pings), wait one second, and post 62 frames of a real face photograph (15 KB each) with a valid signature, starting over three seconds. Because the frames are a still image, liveness fails *after* the full face analysis, so CPU cost approximates a real check-in (detection, blendshapes and up to 12 embeddings) while recording no attendance. Over loopback the RTT is near zero, so any RTT reported is delay from the server being busy. The machine had 20 logical CPUs and 14 GB of memory.

The latency gate was unaffected (maximum RTT 0.9 ms against 20 ms) because analysis runs in worker threads while the event loop keeps answering pings. Response time grows with queue depth, and throughput was about 28–30 per minute from $N = 5$ to $N = 20$ (one run each). The cause is a global lock around the face engine: one clip is analysed at a time. A 2.02 s analysis bounds throughput at about 30 per minute, close to the measured 29. If this holds at larger N, which we did not measure, 60 students arriving together would queue for about two minutes. The test used plain HTTP on loopback, so TLS and the neighbour-table lookup are not included. The nonce is consumed at the start of the request, before analysis, so queueing does not expire it, but the student waits that long for an answer. The test cannot measure radio airtime or the access point's station limit; a monitor script that logs associated stations, leases, per-device ping times and check-in RTTs per interval is provided for use with real devices.

10 Threats and Limitations

- **Sample sizes.** The instrumented Windows trial had three students (17 events) and the Linux session two phones. The 0.363 threshold is the library default, not calibrated on this population. The six-phone MAC observation is informal.
- **Liveness.** The blink count is a minimum, so a pre-recorded video with three blinks passes for $K = 2$ or $K = 3$, and a modified page can upload any frames; blink counting stops still photos, not replay. A randomized head-turn challenge is the planned countermeasure. Blink false rejections are the main usability problem.
- **Identity across the clip.** Identity is checked on up to 12 sampled eyes-open frames, not on every frame; one blurry sampled frame can reject a genuine student.
- **Presence is network membership.** Wi-Fi reaches corridors and neighbouring rooms, and the latency gate cannot see distance within radio range. A student in the corridor passes it; the face check shows who holds the phone, not where.
- **Weaker MAC factor.** Section 8 deliberately lets a MAC change pass when key and face verify. A student can change the MAC without root (the private-address setting or, on an iPhone, forgetting the network), which defeats the one-phone-one-student and 60-second rules; the face check is then the remaining barrier.
- **Capacity.** The Linux access point's station limit is unmeasured and the load test covers up to 20 concurrent check-ins on loopback.
- **Latency estimator.** The trial's RTT figures used the older median of five pings; the minimum-of-ten estimator has one logged live check-in (3.7 ms).
- **Key tied to the portal address.** A browser holds the key per origin, so changing the host's address strands every student's key and forces an instructor reset.
- **State and privacy.** Nonces are held in memory; the dashboard relies on loopback-only access; face embeddings and evidence photographs are stored with no automatic deletion or consent screen; there is no rate limiting on challenges.
- **Host trust.** The host holds the registry and all decisions; compromise of the host compromises the class's records.

11 Prototype Status and Path to Production

AirGated is a prototype. The pipeline is complete and tested for a single room, but "works in trials" is not "ready for production", and turning the first into the second needs time and measurements that have not been made. Table 6 separates what exists from what is missing. We make no schedule claim.

12 Future Work: Dedicated Access Hardware

Both hosts so far are borrowed machines. A Windows hotspot caps clients and owns UDP 53; a Linux laptop uses its only radio for the access point and goes offline. Neither controls where the signal goes. The next step is a purpose-built access unit running an operating-system-independent service, with the aim of controlling the *area* the radio covers and the *direction* it points.

Why hardware. It addresses four findings at once: the client cap of consumer hotspots, the DNS and DHCP conflicts with operating-system services (the unit owns the network stack,

Area	Current state	Needed before production use
Liveness	Blink count at about 12 fps; high false rejection in the September trial	Log per-frame scores, calibrate, raise capture rate, add head-turn challenge
Face threshold	Library default 0.363	Genuine and impostor distributions from the real class under room lighting
Room presence	Network membership only	Radio containment and a site survey per room (Section 12)
Capacity	Face analysis serialized, about 2 s per clip; station limit unmeasured	Parallel or accelerated face pipeline; measured client limit with real phones
Instructor access	Dashboard on the host only	Authenticated instructor access from a phone, rate limiting
Data protection	Embeddings and photos kept indefinitely	Consent, retention and deletion policy; encryption at rest
Operations	Manual startup and scripts; single room	Appliance packaging, updates, multi-room management, monitoring
Assurance	39 tests, mutation checks, few live trials	Security review; trials at class scale (60+ students) across phone models and OS versions

Table 6: What the prototype has and what a production system still needs.

satisfying H1–H5), the gateway requirement behind the MAC lookup, and the lack of a room-presence signal. An earlier plan considered an off-the-shelf dual-band router for the same reasons, including that 5 GHz attenuates faster through walls than 2.4 GHz; it was evaluated but never purchased or tested, and a custom unit generalises that idea.

Coverage control. A sector or directional antenna and an adjustable transmit power can shape the cell so it fills the seating area rather than the corridor (Figure 5). Radio does not respect walls exactly, so each room needs a site survey, and the unit does not stop a second device carried into the room, which the key and face checks still catch. The posture is layered: radio containment as the primary presence boundary, the signed check-in pipeline underneath it, and a brief instructor spot check as a backstop. A received-signal-strength threshold and, where the radio supports fine timing measurement, time-of-flight ranging could add a measured presence signal to the gates; we have not tested either.

Capacity and compute. The unit must support a whole class of associations (the target is a documented, measured client count above the class size, to be verified with the monitor tool and real phones) and must analyse face clips faster than one at a time: the measured 2 s serial analysis is the first limit seen. An accelerator or a parallel pipeline is needed, and analysis time per class should be a stated requirement.

Instructor on a phone. Running a class from the instructor's phone requires an authenticated channel to the unit: the phone enrolled with its own non-extractable key and a face check, a role flag, and rate limiting, so that a student cannot reach the dashboard or reset enrollments. This replaces the present loopback-only rule.

Platform independence and trust. The service should not depend on one vendor's firmware. The unit must keep the same address as the hosts it replaces, because browser-held keys are

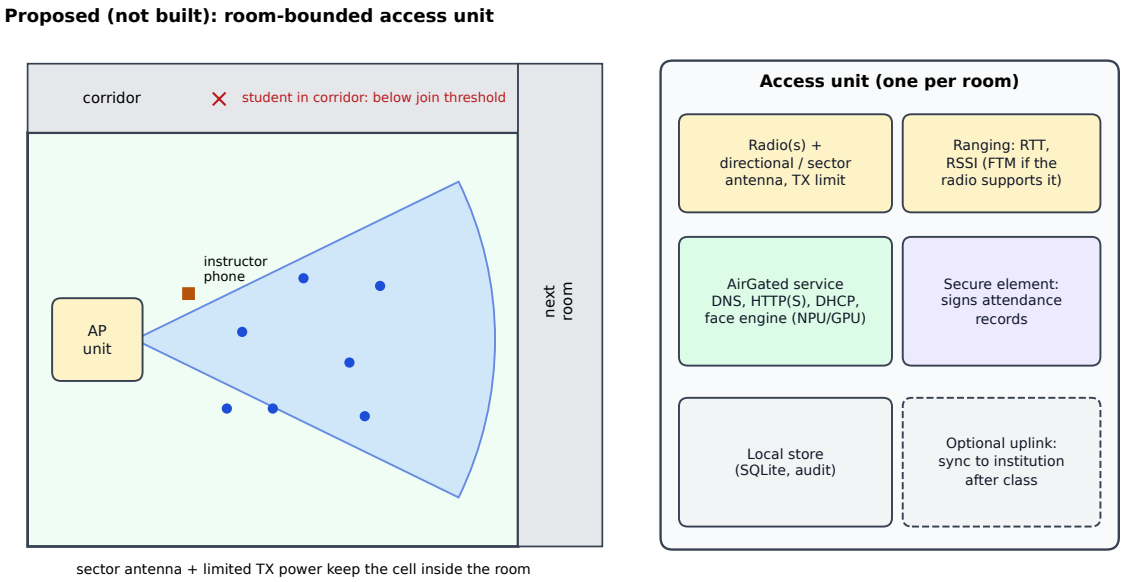


Figure 5: Concept (not built): a per-room unit with a sector antenna and limited transmit power keeps the cell inside the room; a student in the corridor falls below the join threshold. Right: functional blocks of the unit.

bound to the portal's origin; changing it would force every student through an instructor reset. The unit would run the same application on a small embedded Linux image, store records locally, and could sign attendance records with a key held in a secure element and optionally sync them to an institutional system after class. Per-room units would be managed by the institution administrator role of Section 4. None of this is implemented.

13 Conclusion

AirGated shows that a host on the classroom network can bind attendance to the phone (a non-extractable key), the person (a face-and-blink check) and the network (server-measured latency and link-layer address) without cloud services. Its defect history and its move from Windows to Linux show that several hard-to-diagnose failures came from ordering and environment, and that the operating-system-specific surface is small: access-point creation, IP-to-MAC lookup and DNS start order. Treating a changed MAC as a re-registration trigger when key and face verify avoids losing a day's attendance to a changed address, as seen in one live case; blink false rejections remain the main usability problem. The system is still a prototype. The next work is liveness and threshold calibration, class-scale capacity measurement, authenticated instructor access from a phone, and a dedicated access unit that bounds coverage to one room.

Artifacts. Source code, tests, the Linux launcher, hotspot script, monitor, load test and figure generator are in the repository at `https://github.com/unexperiencedone/airgated-`.

Disclosure. The author designed the system, made the design decisions and tested it with real students and phones. Claude (Anthropic's AI coding assistant) was used to help write most of the code, tests and documentation, including this manuscript; the author is responsible for its content. The students who took part agreed to participate and to the use of their face data in testing. Face images are not reproduced and identifiers are truncated.

References

[1] J. Martin, T. Mayberry, C. Donahue, L. Foppe, L. Brown, C. Riggins, E. C. Rye, D. Brown. A study of MAC address randomization in mobile devices and when it fails. *Proceedings on Privacy Enhancing Technologies*, 2017(4):365–383, 2017.

[2] W3C. Web Cryptography API. W3C Recommendation, 2017.

[3] W. Wu, H. Peng, S. Yu. YuNet: A tiny millisecond-level face detector. *Machine Intelligence Research*, 20(5):656–665, 2023.

[4] Y. Zhong, W. Deng, J. Hu, D. Zhao, X. Li, D. Wen. SFace: Sigmoid-constrained hypersphere loss for robust face recognition. *IEEE Transactions on Image Processing*, 30:2587–2598, 2021.

[5] C. Lugaresi et al. MediaPipe: A framework for building perception pipelines. arXiv:1906.08172, 2019.

[6] Android Open Source Project. MAC randomization behavior. `https://source.android.com/docs/core/connect/wifi-mac-randomization-behavior`.

[7] Apple. Use private Wi-Fi addresses on Apple devices. `https://support.apple.com/en-nz/102509`.